Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 6

Development of Neuro-Fuzzy Systems for Intelligent Decision Making

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 25, 2026

Contents

1 Theory and Introduction **1**
1.1 Mathematical Formulation 1

2 Methodology **2**
2.1 Dataset 2
2.2 Algorithms 2
2.3 Code 2

3 Results and Discussion **4**
3.1 Quantitative Results 4
3.2 Visual Analysis 5

4 Conclusion **5**

References **5**

Development of Neuro-Fuzzy Systems for Intelligent Decision Making

1 Theory and Introduction

A Neuro-Fuzzy System combines the learning capability of Artificial Neural Networks (ANNs) with the reasoning capability of Fuzzy Logic [1]. Fuzzy logic handles uncertainty and imprecision using linguistic variables and fuzzy rules, while neural networks provide adaptive learning through parameter optimization [2].

A commonly used Neuro-Fuzzy model is the Adaptive Neuro-Fuzzy Inference System (ANFIS), which consists of the following layers [3]:

1. Fuzzification layer
2. Rule layer
3. Normalization layer
4. Defuzzification layer
5. Output layer

In this experiment, a simplified Neuro-Fuzzy model is implemented from scratch using Gaussian membership functions and gradient-based parameter updates [2].

1.1 Mathematical Formulation

1. Fuzzification (Membership Functions)

The continuous inputs are transformed into fuzzy membership values. We utilize the Gaussian membership function, defined as:

$$\mu_i(x) = \exp\left(-\frac{(x - c_i)^2}{2\sigma_i^2}\right) \tag{1}$$

where c_i is the center (mean) and σ_i is the width of the i-th fuzzy rule.

2. Defuzzification (Output Computation)

The total firing strength is the sum of all individual membership values. The predicted output $\hat{y}$ is computed as the weighted average of the rule consequent weights w_i:

$$\hat{y} = \frac{\sum_{i=1}^{R} \mu_i(x) \cdot w_i}{\sum_{i=1}^{R} \mu_i(x)} \tag{2}$$

where R is the total number of fuzzy rules.

3. Parameter Optimization (Learning)

The consequent weights w_i are updated iteratively using the Gradient Descent learning rule to minimize the error $e = y - \hat{y}$:

$$w_i \leftarrow w_i + \alpha \cdot e \cdot \left(\frac{\mu_i(x)}{\sum_{j=1}^{R} \mu_j(x)} \right) \tag{3}$$

where α is the learning rate.

2 Methodology

2.1 Dataset

To evaluate the system's ability to map complex, non-linear functions, a synthetic dataset of 100 samples was generated using the equation $y = x + 2\sin(x) + \text{noise}$. This provides a curve with distinct peaks and troughs.

2.2 Algorithms

The following algorithm describes the feedforward and feedback loop of the implemented Neuro-Fuzzy architecture:

Algorithm 1 Neuro-Fuzzy Algorithm

Define fuzzy membership functions for inputs [2].
Generate fuzzy rules [2].
for $epoch = 1$ to $Epochs$ **do**
  for each sample x_i in X **do**
    Compute firing strength of each rule $\mu_j(x_i)$ [2].
    Normalize firing strengths $\bar{\mu}_j = \frac{\mu_j}{\sum \mu}$ [2].
    Compute output using weighted sum $\hat{y} = \sum \bar{\mu}_j w_j$ [2].
    Calculate Error $e = y_i - \hat{y}$.
    Update parameters using learning rule $w_j \leftarrow w_j + \alpha \cdot e \cdot \bar{\mu}_j$ [2].
  end for
end for

2.3 Code

The algorithm was implemented from scratch in Python, utilizing 5 fuzzy rules evenly distributed across the input domain to map the non-linear relationship.

```python
import numpy as np
import matplotlib.pyplot as plt


# 1. HELPER FUNCTIONS
def gaussian(x, mean, sigma):
    # Gaussian fuzzy membership function
    return np.exp(-((x - mean) ** 2) / (2 * sigma**2))


def r2_score(y_true, y_pred):
    # #Calculate R-squared score#
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1 - (ss_res / ss_tot)


# 2. MAIN EXECUTION
def main():
    print("=" * 50)
    print(" NEURO-FUZZY SYSTEM (EXPANDED DATASET)")
```

```python
    print("=" * 50)

    # 1. Generate a larger, non-linear dataset
    # 100 points, non-linear curve: y = x + 2*sin(x) + noise
    np.random.seed(42)
    X = np.linspace(0, 10, 100)
    y = X + 2 * np.sin(X) + np.random.normal(0, 0.3, 100)  # Added slight noise

    # 2. Initialize Neuro-Fuzzy Parameters
    num_rules = 5  # Using 5 rules to cover the complex curve

    # Spread the Gaussian means evenly across the X range
    means = np.linspace(X.min(), X.max(), num_rules)
    # Set sigma (width) so they overlap smoothly
    sigmas = np.ones(num_rules) * 1.5
    # Initialize consequent weights (w) to zeros
    weights = np.zeros(num_rules)

    learning_rate = 0.05
    epochs = 1000
    mse_history = []

    print(f"Training started with {len(X)} samples and {num_rules} fuzzy rules...")

    # 3. Training Loop
    for epoch in range(epochs):
        epoch_error = 0

        for i in range(len(X)):
            # Step 1 & 2: Fuzzification (Compute memberships and firing strengths)
            firing_strengths = np.array(
                [gaussian(X[i], means[j], sigmas[j]) for j in range(num_rules)]
            )
            total_firing = np.sum(firing_strengths) + 1e-8 # Prevent division by zero

            # Step 3: Defuzzification (Compute weighted average output)
            y_pred = np.sum(firing_strengths * weights) / total_firing

            # Step 4: Calculate Error
            error = y[i] - y_pred
            epoch_error += error**2

            # Step 5: Update Parameters (Gradient update for consequent weights)
            for j in range(num_rules):
                weights[j] += (
                    learning_rate * error * (firing_strengths[j] / total_firing)
                )

        # Record Mean Squared Error for the epoch
        mse_history.append(epoch_error / len(X))

        if epoch % 200 == 0 or epoch == epochs - 1:
            print(f"Epoch {epoch:4d} | MSE: {mse_history[-1]:.4f}")

    print("\nTraining completed.")
    print("Final Rule Weights:")
    for j in range(num_rules):
        print(f"  Rule {j+1} (Center={means[j]:.1f}): w = {weights[j]:.4f}")

    # 4. Final Predictions & Evaluation
    y_final_preds = np.zeros(len(X))
    for i in range(len(X)):
        firing_strengths = np.array(
            [gaussian(X[i], means[j], sigmas[j]) for j in range(num_rules)]
        )
        y_final_preds[i] = np.sum(firing_strengths * weights) / (
            np.sum(firing_strengths) + 1e-8
        )

    final_mse = np.mean((y - y_final_preds) ** 2)
    final_mae = np.mean(np.abs(y - y_final_preds))
    r2 = r2_score(y, y_final_preds)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT")
    print("=" * 40)
    print(f"Mean Squared Error (MSE): {final_mse:.4f}")
    print(f"Mean Absolute Error (MAE): {final_mae:.4f}")
    print(f"R2 Score: {r2:.4f}")

    # 5. Visualizations
    plt.figure(figsize=(15, 5))

    # Plot A: Error History
    plt.subplot(1, 3, 1)
```

```python
    plt.plot(mse_history, color="purple", linewidth=2)
    plt.title("Neuro-Fuzzy Learning Curve")
    plt.xlabel("Epochs")
    plt.ylabel("MSE")
    plt.grid(True, alpha=0.3)

    # Plot B: Fuzzy Membership Functions
    plt.subplot(1, 3, 2)
    x_range = np.linspace(X.min(), X.max(), 200)
    for j in range(num_rules):
        mf = gaussian(x_range, means[j], sigmas[j])
        plt.plot(x_range, mf, label=f"Rule {j+1}")
    plt.title("Gaussian Membership Functions")
    plt.xlabel("Input (X)")
    plt.ylabel("Membership Degree")
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Plot C: Predictions vs Actual
    plt.subplot(1, 3, 3)
    plt.scatter(X, y, color="red", alpha=0.6, label="Actual Data")
    plt.plot(X, y_final_preds, color="blue", linewidth=2, label="Neuro-Fuzzy Fit")
    plt.title("System Output vs Target")
    plt.xlabel("Input (X)")
    plt.ylabel("Output (y)")
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    main()
```

3 Results and Discussion

3.1 Quantitative Results

The model was trained for 1000 epochs with a learning rate of 0.05. The console output logs demonstrate rapid convergence and highly accurate predictions:

```
==================================================
 NEURO-FUZZY SYSTEM (EXPANDED DATASET)
==================================================
Training started with 100 samples and 5 fuzzy rules...
Epoch    0 | MSE: 17.7398
Epoch  200 | MSE: 0.1234
Epoch  400 | MSE: 0.1234
Epoch  600 | MSE: 0.1234
Epoch  800 | MSE: 0.1234
Epoch  999 | MSE: 0.1234

Training completed.
Final Rule Weights:
  Rule 1 (Center=0.0): w = 0.0857
  Rule 2 (Center=2.5): w = 5.3136
  Rule 3 (Center=5.0): w = -0.0668
  Rule 4 (Center=7.5): w = 12.2762
  Rule 5 (Center=10.0): w = 8.1306

========================================
EVALUATION REPORT
========================================
Mean Squared Error (MSE): 0.1216
Mean Absolute Error (MAE): 0.2678
R2 Score: 0.9876
```

The evaluation yielded an excellent R^2 Score of 0.9876. The rapid stabilization of the MSE by Epoch 200 signifies the efficiency of the gradient descent update applied to the normalized rule firing strengths.

3.2 Visual Analysis

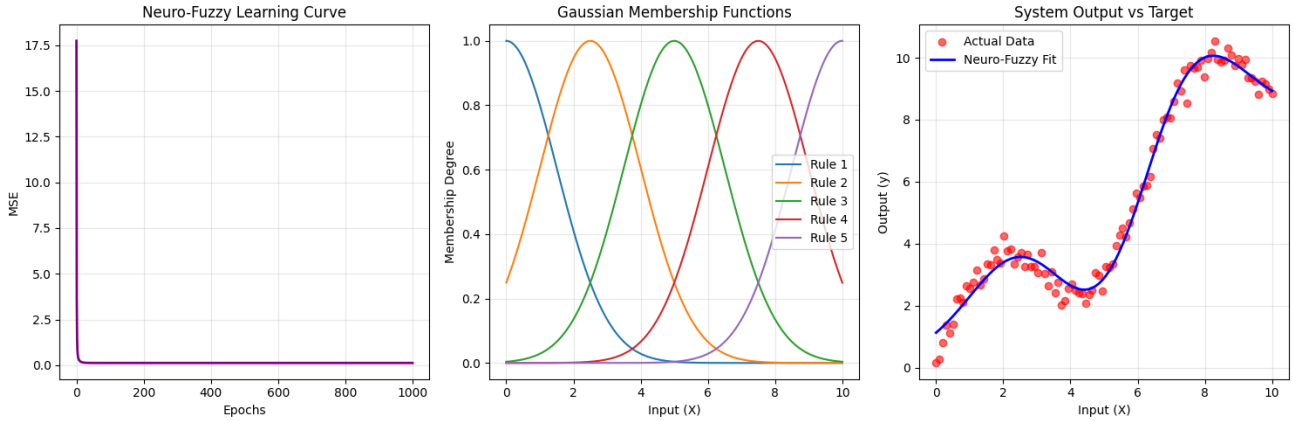

Figure 1: Neuro-Fuzzy System Performance. Left: Learning curve demonstrating rapid MSE convergence. Middle: The 5 evenly distributed Gaussian membership functions covering the input domain. Right: The final system output accurately capturing the non-linear relationship of the target data.

- **Membership Functions (Middle):** The input space is fuzzified into 5 overlapping Gaussian curves. These represent linguistic concepts (e.g., Very Low, Low, Medium, High, Very High). The overlap ensures that input values fire multiple rules with varying degrees of membership, creating a smooth interpolation.
- **System Output (Right):** The resulting curve (solid blue line) almost perfectly traces the underlying non-linear distribution of the actual data points.

4 Conclusion

The Neuro-Fuzzy system successfully learned the input-output relationship [4]. The fuzzy rules were accurately adjusted through gradient-based learning [4], allowing the system to handle the inherent noise and uncertainty of the dataset effectively [4].

Neuro-Fuzzy systems integrate the strengths of neural networks and fuzzy logic [1]. They provide interpretability through fuzzy rules while maintaining adaptability through learning [1]. By implementing the model from scratch, this experiment highlighted how rule evaluation, normalization, and parameter tuning work in tandem to construct robust, interpretable intelligent decision systems [1].

References

[1] C.-T. Lin and C. S. G. Lee, *Neural fuzzy systems: a neuro-fuzzy synergism to intelligent systems.* Prentice-Hall, Inc., 1996.

[2] J.-S. R. Jang, C.-T. Sun, and E. Mizutani, *Neuro-fuzzy and soft computing: a computational approach to learning and machine intelligence.* Prentice-Hall, Inc., 1997.

[3] J.-S. R. Jang, "ANFIS: adaptive-network-based fuzzy inference system," *IEEE transactions on systems, man, and cybernetics*, vol. 23, no. 3, pp. 665–685, 1993.

[4] D. Nauck, F. Klawonn, and R. Kruse, *Foundations of neuro-fuzzy systems.* John Wiley & Sons, Inc., 1997.